

```

import os
import json
import glob
import requests
from datetime import datetime, timezone, timedelta
from jinja2 import Environment, FileSystemLoader
from playwright.sync_api import sync_playwright

def categorize_vocabulary(vocab_items):
    """Sorts vocabulary into universal categories across all articles."""
    categorized = {
        "core_vocab": [], "one_word_subs": [], "fixed_prepositions": [], "phrasal_verbs": [], "idioms": [], "foreign_words": []
    }
    for item in vocab_items:
        cat = str(item.get("category", "")).strip().lower()
        if "one-word" in cat or "one word" in cat:
            categorized["one_word_subs"].append(item)
        elif "preposition" in cat:
            categorized["fixed_prepositions"].append(item)
        elif "phrasal" in cat:
            categorized["phrasal_verbs"].append(item)
        elif "idiom" in cat:
            categorized["idioms"].append(item)
        elif "foreign" in cat:
            categorized["foreign_words"].append(item)
        else:
            categorized["core_vocab"].append(item)
    return categorized

def send_to_telegram(pdf_path, date_range_formatted, editorial_titles):
    bot_token = os.getenv("TELEGRAM_BOT_TOKEN")
    chat_id = os.getenv("ADMIN_CHAT_ID") or os.getenv("TELEGRAM_CHAT_ID")
    if not bot_token or not chat_id:
        print("⚠️ Telegram BOT_TOKEN or ADMIN_CHAT_ID missing. Skipping Telegram upload.")
        return
    quote_lines = [f"{{idx:02d}} {{title}}" for idx, title in enumerate(editorial_titles, start=1)]
    quote_content = "\n".join(quote_lines)
    caption = (f"📖 Ez Editoriális Weekly Compilation ({{date_range_formatted}})\n" f"{{quote_content}}")

    url = f"https://api.telegram.org/bot{bot_token}/sendDocument"
    filename = os.path.basename(pdf_path)
    print(f"📁 Uploading {filename} to Telegram...")
    with open(pdf_path, "rb") as doc:
        files = {"document": (filename, doc, "application/pdf")}
        payload = {"chat_id": chat_id, "caption": caption, "parse_mode": "HTML"}
        res = requests.post(url, data=payload, files=files)
        if res.status_code == 200:
            print(f"🚀 Successfully sent Weekly Compilation to Telegram!")
        else:
            print(f"❌ Telegram API Error ({{res.status_code}}): {{res.text}}")

def compile_weekly_magazine():
    base_dir = os.path.dirname(os.path.abspath(__file__))
    backups_dir = os.path.join(base_dir, "backups")
    templates_dir = os.path.join(base_dir, "templates")
    build_dir = os.path.join(base_dir, "build")
    output_dir = os.path.join(base_dir, "output")
    os.makedirs(build_dir, exist_ok=True)
    os.makedirs(output_dir, exist_ok=True)
    json_files = sorted(glob.glob(os.path.join(backups_dir, "*.json")))
    if not json_files:
        print(f"⚠️ No backup JSON files found in: {{backups_dir}}")
        return
    aggregated_editorials = []
    all_raw_dates = []
    for file_path in json_files:
        with open(file_path, "r", encoding="utf-8") as f:
            try:
                daily_data = json.load(f)
                aggregated_editorials.extend(daily_data.get("editorials", []))
            except:
                # Extract date from json metadata or filename
                scraped_date = daily_data.get("date_scraped")
                if scraped_date:
                    all_raw_dates.append(scraped_date)
            except json.JSONDecodeError:
                print(f"⚠️ Skipping corrupted JSON: {{file_path}}")
    # Enforce maximum of 25 editorials
    aggregated_editorials = aggregated_editorials[:25]
    # Calculate date range from Monday to Saturday
    all_raw_dates = sorted(list(set(all_raw_dates)))
    if all_raw_dates:
        start_dt = datetime.strptime(all_raw_dates[0], "%Y-%m-%d")
        end_dt = datetime.strptime(all_raw_dates[-1], "%Y-%m-%d")
        date_range_formatted = f"{{start_dt.strftime('%d %b')}} – {{end_dt.strftime('%d %b %Y')}}"
    else:
        now = datetime.now(timezone(timedelta(hours=5, minutes=30)))
        date_range_formatted = now.strftime("%d %b %Y")
    # Collect unique newspapers and build TOC index entries
    newspapers = set()
    index_entries = []
    editorial_titles = []
    all_vocab_items = []
    for art in aggregated_editorials:
        title = art.get("title", "Untitled Editorial")
        np = art.get("newspaper", "Editorial")
        editorial_titles.append(title)
        newspapers.add(np)
        index_entries.append({"title": title, "newspaper": np, "timestamp": art.get("timestamp", "")})
    # Accumulate vocab items for universal grouping
    all_vocab_items.extend(art.get("editorial_vocabulary", []))
    # Universal Categorization across all 25 articles
    universal_vocab = categorize_vocabulary(all_vocab_items)
    newspapers_covered = "& ".join(sorted(newspapers))
    if newspapers:
        newspapers_covered = "National Dailies"
    # Render Template
    env = Environment(loader=FileSystemLoader(templates_dir))
    template = env.get_template("weekly_template.html")
    rendered_html = template.render(
        date_range_formatted=date_range_formatted,
        total_articles=len(aggregated_editorials),
        total_words=len(all_vocab_items),
        newspapers_covered=newspapers_covered,
        index_entries=index_entries,
        universal_vocab=universal_vocab
    )
    rendered_html_path = os.path.join(build_dir, "weekly_magazine.html")
    with open(rendered_html_path, "w", encoding="utf-8") as f:
        f.write(rendered_html)
    # Output PDF
    ist_time = datetime.now(timezone(timedelta(hours=5, minutes=30)))
    pdf_filename = f"Weekly_Compilation_{{ist_time.strftime('%Y-%m-%d')}}.pdf"
    output_pdf_path = os.path.join(output_dir, pdf_filename)
    with sync_playwright() as p:
        browser = p.chromium.launch(args=["--no-sandbox", "--disable-setuid-sandbox"])
        page = browser.new_page()
        page.goto(f"file://{{rendered_html_path}}", wait_until="networkidle")
        page.pdf(
            path=output_pdf_path,
            format="A4",
            print_background=True,
            margin={"top": "0mm", "bottom": "0mm", "left": "0mm", "right": "0mm"}
        )
        browser.close()
    print(f"✅ Generated Weekly Magazine: {{output_pdf_path}}")
    send_to_telegram(output_pdf_path, date_range_formatted, editorial_titles)
    if __name__ == "__main__":
        compile_weekly_magazine()

```